



MERRIMACK COLLEGE

CSC 6303

Week 4

From Python to JavaScript and Perl

Systems and Languages Survey - Dr. Paulo Fernandes

Presentation Agenda

Week 4

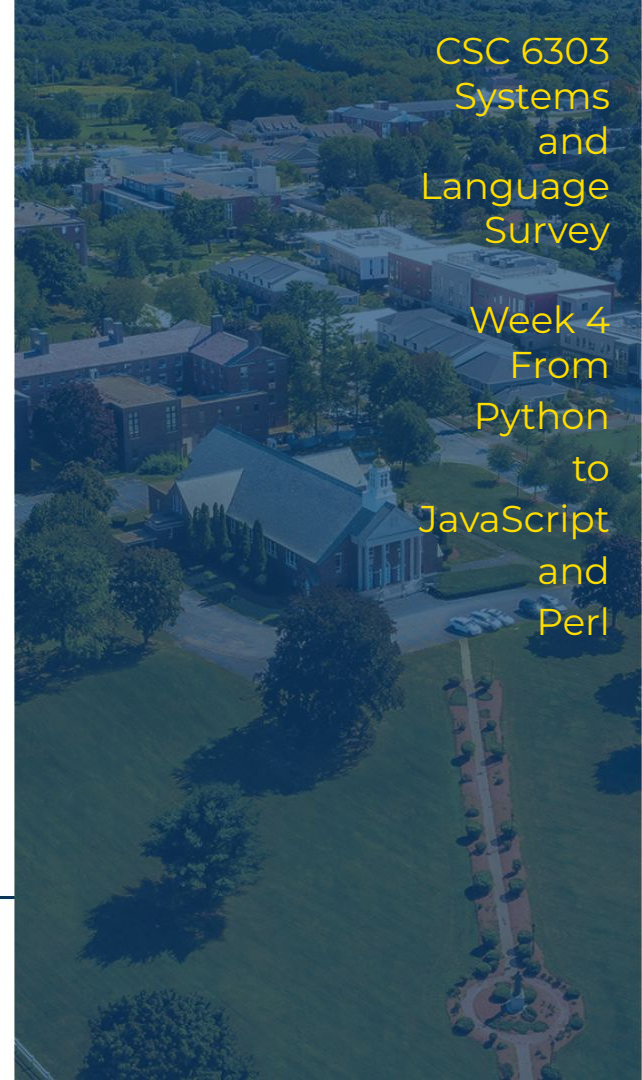
- Using JavaScript and Perl
 - a. Online or With and IDE
- JavaScript and the WWW
 - a. Purpose
 - b. Basic Commands
 - c. JavaScript and HTML/CSS
- Perl 5
 - a. Purpose
 - b. Basic Commands
- This Week's tasks



MERRIMACK COLLEGE

CSC 6303
Systems
and
Language
Survey

Week 4
From
Python
to
JavaScript
and
Perl



Using JavaScript and Perl

You can use it online, but it may be installed in your own machine through an IDE, like VScode.

Likely your machine has either JavaScript available through your browser. Perl, however, is less likely already in your machine. Both languages, though are easily installed in all platforms.

A common characteristics of both languages is their nature that in the beginning were created in a similar approach as Python, just providing a script language for automating small tasks.



Running code online

Among the several online options you can try these ones:

You can try it on one of the following
JavaScript interpreters online:

[Replit](#) - [JDoodle](#) - [W3Schools](#)

You can try the following Perl
interpreter online:

[JDoodle](#)

A curious thing is that such online servers always behave as interpreters, but refer to themselves as compilers, probably due to their linking functionalities.



MERRIMACK COLLEGE

Friendly advice, using an online server as such, always
keep a local copy of your code.

JavaScript and the WWW

An *ad hoc* language to
provide some intelligence
to the World Wide Web.

The first important thing about JavaScript is that it is not really related to Java, except by name, and by being a procedural language inspired by Algol and C.

The JavaScript was created within Netscape company, to serve as intelligence to the web browsing experience, initially provided by HTML, then CSS, and at last JavaScript.

- Purpose
- Basic Commands
- JavaScript and HTML/CSS



History and Purpose

JavaScript was created in 1995 within Netscape, a company created by the developers of the first graphical browser, Mosaic, which was succeeded by a commercial and more sophisticated browser, the Netscape Navigator.



The basic technology behind the WWW (World Wide Web) was the markup language HTML to transform text into hypertext. Accompanying the HTML (Hypertext Markup Language) was available the declarative language CSS (Cascade Style Sheets) providing document formatting, but soon as the WWW was evolving, JavaScript was created to provide scripting definition of web features. As such, JavaScript purpose was modest, just offer some simple responsiveness to hypertext documents.



Program Structure

- In **Python**
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes.
- In **JavaScript**
 - Often scripts, frequently embedded functions, sometimes functions to call, and rarely classes with methods.



```
// Bubble Sort in JavaScript
function bubbleSort(arr) {
  for(var i = 0; i < arr.length; i++) {
    for(var j = 0; j < ( arr.length - i -1 ); j++) {
      if(arr[j] > arr[j+1]) {
        var temp = arr[j]
        arr[j] = arr[j + 1]
        arr[j+1] = temp
      }
    }
  }
  console.log(arr);
}

var arr = [64, 34, 25, 12, 22, 11, 90];
bubbleSort(arr);
```

C++
similar
style
{};

A script



Basic Commands - functions

- In Python - **def**
 - functions are untyped
 - input parameters can be omitted/default values
 - simple variables passed by value, lists passed by reference (almost always)
 - none, single, or multiple untyped return
- In JavaScript - **function**
 - functions are untyped
 - input parameters passed always by value
 - single return value (can be an array - JavaScript equivalent of Python lists - or an object)
 - untyped variable declaration

```
def sumMinMax(a, b):  
    sum = a + b  
    if a < b:  
        min, max = a, b  
    else:  
        min, max = b, a  
    return sum, min, max
```



```
function sumMinMax(a, b) {  
    let sum = a + b;  
    let min, max;  
    if (a < b) {  
        min = a;  
        max = b;  
    } else {  
        min = b;  
        max = a;  
    }  
    return [sum, min, max];  
}
```



Basic Commands - variables

```
let name = "John Addams";    // String
let age = 90;                // Number
let isPresident = true;      // Boolean
let occupations = [         // Array
    1758, 'Lawyer',
    1774, 'Politician',
    1778, 'Diplomat'];
let president = {           // Object
    name: "John Addams",
    age: 61,
    inauguration: 1797
};
```



- In Python
 - Integers, Floating points, Booleans, and Lists (including Strings).
 - No declaration or allocation needed.
- In JavaScript
 - Integers (limited precision), Floating points, Booleans, Strings, Arrays (which in JavaScript can be of different types, like Python lists), and Objects.
 - Declaration needed (**let** or **var**), untyped and implicit allocation.

Text: [Variables in JavaScript.](#)



MERRIMACK COLLEGE

Text: [Data Types in JavaScript.](#)

Basic Commands - decision

- In Python
 - if - elif - else
- In JavaScript
 - if - else
 - switch - case - default
 - switch (true) and case condition
 - conditional operator

```
switch (year) {  
  case 1770:  
  case 1771:  
  case 1772:  
  case 1773:  
  case 1774:  
  case 1775:  
    alert( 'Too early...' );  
    break;  
  case 1776:  
    alert( 'Exactly!' );  
    break;  
  default:  
    alert( 'Too late...' );  
}
```

```
if expression1:  
    statement(s)  
elif expression2:  
    statement(s)  
elif expression3:  
    statement(s)  
else:  
    statement(s)
```



```
if (year < 1776) {  
  alert( 'Too early...' );  
} else if (year > 1776) {  
  alert( 'Too late...' );  
} else {  
  alert( 'Exactly!' );  
}
```

```
let message = (year < 1776) ? 'Too early...' : (year > 1776) ? 'Too late...' : "Exactly!";  
alert(message);
```

```
(year < 1776) ? alert('Too early...') : (year > 1776) ? alert('Too late...') : alert('Exactly!')
```



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In JavaScript
 - **for** loops (break - continue)
 - **while** loops (break - continue)
 - **do while** loops (break - continue)

```
fruits = ['banana', 'apple', 'mango']  
for index in range(len(fruits)):  
    print 'Current fruit :', fruits[index]
```

```
count = 0  
while (count < 9):  
    print 'The count is:', count  
    count = count + 1
```



```
for (let i = 0; i < 10; i++) {  
    alert(i);  
}
```

```
let i = 0;  
while (i<10) {  
    alert(i);  
    i++;  
}
```

```
let i = -1;  
do {  
    i++;  
    alert(i);  
} while (i<10);
```



OO concepts in JavaScript

The Object-Oriented concepts have been patched up into JavaScript, but not many programs use them, rather sticking with just objects, a native data type.

JavaScript provides some basic forms of **inheritance** through Object creation, which can also be used to provide **polymorphism**.

Those concepts can be achieved without properly using classes, in a similar manner to Go lang approach.



```
let animal = {
  name: 'Animal',
  sayName: function() {
    console.log(this.name);
  }
};

let dog = Object.create(animal);
dog.name = 'Dog';
dog.sound = function() {
  console.log('Woof!');
};

let cat = Object.create(animal);
cat.name = 'Cat';
cat.sound = function() {
  console.log('Meow!');
};

dog.sayName(); // logs 'Dog'
dog.sound();   // logs 'Woof!'
cat.sayName(); // logs 'Cat'
cat.sound();   // logs 'Meow!'
```



OO concepts in JavaScript

Encapsulation is optional for instance variable and methods by assigning identifiers starting with a underscore sign.

The definition of classes represents a better programming style, than using Objects (simple data structures), but it is rarely employed in JavaScript embedded into web pages.



```
class President {
  constructor(name, age, inauguration) {
    this.name = name;
    this._age = age;
    this._inauguration;
  }
  getAge()          { return this._age; }
  getInauguration() { this._inauguration = inauguration; }
}

var president = new President("John Addams", 90, 1797);
console.log(president.name);           // outputs "John Addams"
console.log(president.getAge());       // outputs 90
console.log(president.getInauguration()); // outputs 1797
```



JavaScript within HTML/CSS



The original, and largely the more common, use of Javascript is to provide scripts for web pages.

A web page according to World Wide Web Consortium (W3C) can be defined by an HyperText Markup Language (HTML) file that may contain commands or refer to a Cascade Style Sheet (CSS) file, and JavaScript commands (embedded or referred).

```
<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript Example</title>
  </head>
  <body>
    <h1 id="header">Waiting for you to click...</h1>
    <button id="button">Click Me</button>
    <script>
      const header = document.querySelector("#header");
      const button = document.querySelector("#button");
      button.addEventListener("click", function() {
        header.innerHTML = "Waiting for you to click... You clicked!";
      });
    </script>
  </body>
</html>
```

Waiting for you to click...

Click Me



Waiting for you to click... You clicked!

Click Me



MERRIMACK COLLEGE

Video: [JavaScript Basics](#).

HTML/CSS 101



HTML modules:

- The language version
 - **<!DOCTYPE html>**
- The whole content
 - **<html> ... </html>**
- The page header section
 - **<head> ... </head>**
 - **<title> ... </title>**
- The body section
 - **<body> ... </body>**
 - **<h1>, <h2>, ..., <h6>**
 - **<p>,
**

```
<!DOCTYPE html>
<html>
  <head>
    <title>HTML Example</title>
  </head>
  <body>
    <h1 id="header">This is a first level header</h1>
    <p>This is a common paragraph, only extremely longer than it need to be.
    It is not to boring you reading it, just to be able to show you that the
    line break has not to do in the source and in the rendered web page.
    You can try to reshape your browser and see how it rewrap your paragraph.
    In fact, the only part that remains breaking is the breaking line command
    (br) of the markup language. <br> This is after the line break.
    </p>
    <h2>This is a second level header</h2>
    <p>This is a first paragraph below the second header.
    Which is also extended to try to make a implicit line break.</p>
    <p>This is the second paragraph within the second level header.</p>
  </body>
</html>
```



MERRIMACK COLLEGE

Download: exemp.html (save it and double click it).

HTML/CSS/JS basics



HTML commands within the body:

- **<style>**
 - defines style sheet (CSS) directives
- **<button>**
 - creates a button, and when clicked take an action
- **<script>**
 - defines a function in JavaScript

```
<!DOCTYPE html>
<html>
  <head>
    <title>Button and Style</title>
    <style>
      h2 {color: blueviolet}
      body {background-color: beige; color: grey}
    </style>
  </head>
  <body>
    <h2>Click to change</h2>
    <p>Style changes can be done using a change in attributes</p>
    <p>In this case it changes the style of the level 3 header.</p>
    <h3 id="change_style"; style="color: blue">This Text Changes Color by Attribute</h3>
    <button onclick="document.getElementById('change_style').style.color='green'">Green</button>
    <button onclick="document.getElementById('change_style').style.color='red'">Red</button>
    <button onclick="document.getElementById('change_style').style.color='blue'">Blue</button>
    <p>Other properties, and also color, can be changed using a JavaScript function.</p>
    <h3 id="change_js"; style="color: darkorchid">This Text Changes Color by JavaScript</h3>
    <script>
      function changeColor(color) {
        if (color == 1) { document.getElementById("change_js").style.color = 'black'; }
        else if (color == 2) { document.getElementById("change_js").style.color = 'orange'; }
        else { document.getElementById("change_js").style.color = 'Orchid'; }
      }
    </script>
    <button onclick="changeColor(1)">Black</button>
    <button onclick="changeColor(2)">Pink</button>
    <button onclick="changeColor(3)">Orange</button>
  </body>
</html>
```



MERRIMACK COLLEGE

Download: [change.html](#) (save it and double click it).

HTML/CSS/JS input/output



HTML commands within the body:

- **<label>**
 - just a text label
- **<input>**
 - defines an input box:
 - type="text"
 - type="number"
- **.innerHTML**
 - the text of an <p> element

```
<!DOCTYPE html>
<html>
<head>
  <title>Input and Output Example</title>
  <script>
    function displayPrompt() {
      var name = document.getElementById("inputName").value;
      var age = document.getElementById("inputAge").value;
      if (name != "") {
        document.getElementById("outputName").innerHTML = "Hello, " + name;
        document.getElementById("outputName").style.color = "red";
        if (age < 30) { document.getElementById("outputAge").innerHTML = "You are very young!"; }
        else { document.getElementById("outputAge").innerHTML = "You are not too old to learn!"; }
        document.getElementById("outputAge").style.color = "blue"; }
      else { document.getElementById("outputName").innerHTML = "... still waiting ..."; }
    }
  </script>
</head>
<body>
  <h1 style="color:red">Getting to know you</h1>
  <label>Enter Your Name:</label>
  <input type="text" id="inputName"> <br>
  <label>Enter Your Age:</label>
  <input type="number" id="inputAge"> <br> <br>
  <button onclick="displayPrompt()">Submit</button>
  <p id="outputName">I'm waiting...</p>
  <p id="outputAge"></p>
</body>
</html>
```



More on JavaScript



- JavaScript was meant to provide basic script capabilities to web pages, but it is a full language in itself, capable to implement any sort of algorithm.
- A JavaScript code is usually composed by script functions that may be used by an HTML page.
 - It is possible to represent the style sheets (CSS) outside of the HTML file, as well as the JavaScript functions in a separated .js file.
- Mobile development is based on specific techniques to develop reliable, correct, and efficient HTML/CSS/JS sets of appropriated files.
- Most browsers, if not all, tolerate mistakes in HTML, CSS, and JS files, but how they handle those errors may differ considerably
 - The W3C defines standards that all browsers should comply.



Perl 5

Perl is an old script language that has evolved considerably and actually bifurcate into two languages: Perl 5 and Perl 6 (since 2019 known as Raku).

Perl offers alternatives to script processing and it is frequently employed to system administration, network programming, and other applications, such as for GUIs.

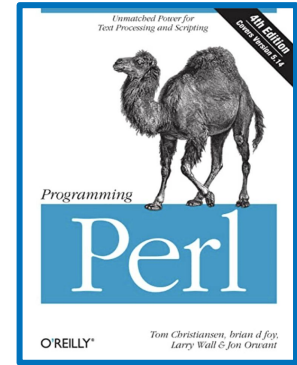
There is a reasonable amount of legacy code in Perl, but the language evolution relies much on the future language evolution that is expected to released version 7 in the near future.

- Purpose
- Basic Commands



History and Purpose

Created in 1987 within Unisys and it evolved rapidly at the beginning, becoming rather stable since the turn of century. In 2019, a parallel version Perl 6 was launched, while the on going version 5.26 remained operational.



Perl language is considerably compact and reputed as a hacking language employed by system programmers. It is important to call the attention that Perl language originally is almost as old as C language.

- One of the goals of the language was to provide a platform independent language to serve several operating systems (which were more abundant by the 80's and 90's).



Program Structure

- In **Python**
 - Mostly packages, sometimes scripts, sometimes a main function, sometimes classes.
- In **Perl**
 - Several packages, mostly subroutines (functions), scripts, sometimes classes.



```
# Bubble sort in Perl 5
sub bubble_sort {
    my (@array) = @_;
    my $n = scalar(@array);
    for (my $i = 0; $i < $n - 1; $i++) {
        for (my $j = 0; $j < $n - $i - 1; $j++) {
            if ($array[$j] > $array[$j + 1]) {
                ($array[$j], $array[$j + 1]) = ($array[$j + 1], $array[$j]);
            }
        }
    }
    return @array;
}

my @array = (64, 34, 25, 12, 22, 11, 90);
@array = bubble_sort(@array);
print join(', ', @array), "\n";
```

C
similar
style
{};

older
syntax



Basic Commands - functions

- In Python - **def**
 - functions are untyped
 - input parameters can be omitted/default values
 - simple variables passed by value, lists passed by reference (almost always)
 - none, single, or multiple untyped return
- In Perl - **sub**
 - functions are untyped
 - input parameters untyped passed either by value or reference (user choice)
 - none, single, or multiple untyped return
 - no function overload

```
def sumMinMax(a, b):  
    sum = a + b  
    if a < b:  
        min, max = a, b  
    else:  
        min, max = b, a  
    return sum, min, max
```



```
sub sumMaxMin {  
    my ($a, $b) = @_;  
    my $sum = $a + $b;  
    my ($min, $max);  
    if ($a < $b) {  
        $min = $a;  
        $max = $b;  
    } else {  
        $min = $b;  
        $max = $a;  
    }  
    return ($sum, $min, $max);  
}
```



Basic Commands - variables & data types

- In Python - untyped
 - Integers, Floating points, Booleans, and Lists (including Strings)
 - No declaration or allocation needed
- In Perl - untyped
 - Numbers and Strings (scalars), Arrays, Hashes, and References
 - scalars, denoted by **\$**
 - arrays, denoted by **@**
 - hashes, denoted by **%**
 - references, denoted by ****
 - Special variables (predefined **\$_** ...)



```
$name = "John Addams";      # scalar
$age = 90;                  # scalar
@occupations_date = (      # array
    1758, "Lawyer",
    1774, "Politician",
    1778, "Diplomat"
);
%president = (              # hash
    name => "John Addams",
    age => 61,
    inauguration => 1797
);
$potus2 = \%president       # a reference
print $potus2->{name};      # it prints John Addams
```



Basic Commands - decision

- In Python
 - if - elif - else
- In Swift
 - if - elsif - else
 - unless - elsif - else
 - conditional operator



```
if ($num < 0) {  
    print "$num is negative\n";  
} elsif ($num == 0) {  
    print "$num is zero\n";  
} else {  
    print "$num is positive\n";  
}
```

```
if expression1:  
    statement(s)  
elif expression2:  
    statement(s)  
elif expression3:  
    statement(s)  
else:  
    statement(s)
```



```
$result = ($num < 0) ? "negative" : ($num == 0) ? "zero" : "positive";
```

```
unless ($num < 0)  
    print "$num has rational roots";  
else  
    print "$num has imaginary roots";
```

- given - when - default

```
given ($value) {  
    when (1) { print "The value is 1\n"; }  
    when (2) { print "The value is 2\n"; }  
    when (3) { print "The value is 3\n"; }  
    default { print "The value is not 1, 2, or 3\n"; }  
}
```



Basic Commands - loops

- In Python
 - for loop - range - break - continue - pass - else
 - while loop - break - continue - else
- In Perl
 - for in loops
 - foreach loops
 - while loops
 - do while loops
 - until loops
 - last (equivalent to break in C)
 - next (equivalent to continue in C)

```
fruits = ['banana', 'apple', 'mango']
for index in range(len(fruits)):
    print 'Current fruit :', fruits[index]
```

```
count = 0
while (count < 9):
    print 'The count is:', count
    count = count + 1
```

```
for ($i = 0; $i < 10; $i++)
    print "$i\n";
```

```
$count = 0;
do {
    print "$count\n";
    $count++;
} while ($count < 10);
```

```
$count = 0;
until ($count >= 10) {
    print "$count\n";
    $count++;
}
```

```
@arr = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
foreach $arr (@arr)
    print "$arr\n"
```

```
$count = 0;
while (1) {
    print "$count\n";
    $count++;
    if ($count >= 10) {
        last;
    }
}
```



Object Oriented Concepts in Perl

Object Oriented concepts in Perl are similar to Python by:

- Definition of subclasses for inheritance;
- Definition of methods and constructors;
- Definition of instance variables.

However, they differ in terms of:

- Encapsulation choices in Perl;
- Polymorphism with functions passed as parameters;
- Classes definitions are implicitly bounded.

```
package Dog;
use base 'Animal';
sub new {
    my ($class) = @_;
    my $self = $class->SUPER::new();
    $self->{breed} = 'Unknown';
    bless $self, $class;
    return $self;
}
sub sound {
    my ($self) = @_;
    print "Dog is barking\n";
}
```

```
package Animal;
sub new {
    my ($class) = @_;
    my $self = {};
    bless $self, $class;
    return $self;
}
sub sound {
    print "Animal is making a sound\n";
}
```



This Week's tasks

- Post discussion
- Programming project #4
- Quiz #2

Tasks

- Comment on **which language** among the ones seen so far you found **more diverse** and **why**.
- Create an interactive web page using **JavaScript**.
- Take (available Saturday) **Quiz #2** about the contents of **Week 3** and **Week 4**.



Post Discussion - Week 4 - In-class exercises

Comment on which language among the ones seen so far you found more diverse, and why.

- Python, C, C++, Java, Go, Swift, JavaScript, and Perl, which one you think is the oddball in this set? Which one you think will be more awkward for you to adapt to use? State your reasons to justify your choices.
 - Feel free to give personal reasons, but relate those with technical characteristics of the language.

Your task:

- Post your discussion in the message board until this Saturday;
- Reply posts of your colleagues in the message board until next Wednesday.



MERRIMACK COLLEGE

This task counts towards the In-Class Exercises grade.

Progr. Project #4 - Week 4 - P#4

Create an interactive web page using JavaScript.

Adapt the web page seen in slide 18 ([in_out.html](#)), to create a web page that receives a number between 2 and 9999 and answers if the number entered is a prime number or not.

Base your solution in the [C++ code](#) from Week 2.

```
<!DOCTYPE html>
<html>
<head>
  <title>Prime Checker</title>
  <script>
    function isPrime() { ...
  </script>
</head>
<body>
  <h1 style="color: red">Prime Checker</h1>
  <label>Enter a number from 2 to 9999:</label>
  <input type="number" id="nbr"> <br> <br>
  <button onclick="isPrime()">Submit</button>
  <p id="outputAnswer">I'm waiting...</p>
</body>
</html>
```

Prime Checker

Enter a number from 2 to 9999: 3459

Submit

This number is not prime!

Prime Checker

Enter a number from 2 to 9999: 3457

Submit

This number is prime!



Progr. Project #4 - Week 4 - P#4

- You must submit only the HTML file containing the JavaScript code (if multiple files zip it in a single file).
- If you need to add any explanation or messages to me, make it in the code remarks.
- The quality of your code remarks is not relevant for the evaluation, but make sure you write your name in the remarks.
- This C++ code of the program prime.cpp is available [here](#), the model file in_out.html is [here](#).

Your task:

- Save your file and submit it in the appropriate Canvas until Next Wednesday.

This assignment counts towards
the Coding Projects grade.



MERRIMACK COLLEGE

Quiz #2 - Available this Saturday

- This quiz covers the contents presented in Week 3 and Week 4.
- The quiz is not timed, but the hard deadline to submit it is next Wednesday.
- The quiz is composed by 10 simple questions that you need to answer by choosing the correct answer.
- You can only submit the quiz once, so make sure you double checked your answers before submitting it.
- This quiz is open notes and you can consult any materials referenced in these slides to answer it, but the answer must be your own, not from others.

Your task:

- Submit your quiz in Canvas until Next Wednesday 11:59pm EST.



MERRIMACK COLLEGE

This assignment counts towards the Quizzes grade.

” Week 4 of CSC 6303

- **Post in the discussions until Saturday, reply until next Wednesday**
 - **Do the coding project until next Wednesday**
 - **Take Quiz #2 until next Wednesday**
-

Next Week - Operating Systems basic concepts and organization



MERRIMACK COLLEGE